



PAGINATION DE LA BASE DE DONNÉES



Introduction

Dans d'autres frameworks, la pagination peut être très pénible. Nous espérons que l'approche de Laravel en matière de pagination sera une bouffée d'air frais. Le paginateur de Laravel est intégré au constructeur de requêtes et à l'ORM Eloquent et permet une pagination pratique et facile à utiliser des enregistrements de la base de données, sans aucune configuration.

Pagination des résultats du QueryBuilder

Il existe plusieurs façons de paginer les éléments. La plus simple est d'utiliser la méthode **paginate** du constructeur de requêtes ou d'une requête Eloquent. La méthode **paginate** se charge automatiquement de définir la "**limite**" et le "**décalage**" de la requête en fonction de la page courante visualisée par l'utilisateur. Par défaut, la page actuelle est détectée par la valeur de l'argument de la chaîne de requête de la page dans la requête HTTP. Cette valeur est automatiquement détectée par Laravel, et est également automatiquement insérée dans les liens générés par le paginateur.

Pagination des résultats du QueryBuilder

Dans cet exemple, le seul argument transmis à la méthode **paginate** est le nombre d'éléments que vous souhaitez afficher "par page". Dans ce cas, précisons que nous souhaitons afficher 15 éléments par page :

```
public function index()
{
    return view('user.index', [
        'users' => DB::table('users')->paginate(15)
    ]);
}
```

Pagination simple

La méthode **paginate** compte le nombre total d'enregistrements correspondant à la requête avant de récupérer les enregistrements dans la base de données. Cela permet au paginateur de savoir combien de pages d'enregistrements il y a au total. Toutefois, si vous ne prévoyez pas d'afficher le nombre total de pages dans l'interface utilisateur de votre application, la requête de comptage des enregistrements est inutile.

Pagination simple

Par conséquent, si vous n'avez besoin que d'afficher de simples liens "**Suivant**" et "**Précédent**" dans l'interface utilisateur de votre application, vous pouvez utiliser la méthode **simplePaginate** pour effectuer une requête unique et efficace :

```
$users = DB::table('users')->simplePaginate(15);
```

Affichage des résultats de la pagination

Les instances du paginateur sont des itérateurs et peuvent être bouclées comme un tableau.

```
<div class="container">
@foreach ($users as $user)
{{ $user->name }}
@endforeach
</div>

{{ $users->links() }}
```

La méthode **links** rendra les liens vers le reste des pages de l'ensemble de résultats. Chacun de ces liens contiendra déjà la variable appropriée de la chaîne de requête de la page.